Docker Installation (Follow the instruction up to step 4)

https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04

```
mlops@sujan-Latitude-7490:~$ docker
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login       Authenticate to a registry
  logout      Log out from a registry
  search      Search Docker Hub for images
  version     Show the Docker version information
```

You need to execute all(one by one) the commands and queries highlighted in bold.

sudo systemctl status docker

`sudo systemctl enable/disable docker`(Start Automatically at Boot/Do Not Start at Boot)

docker version

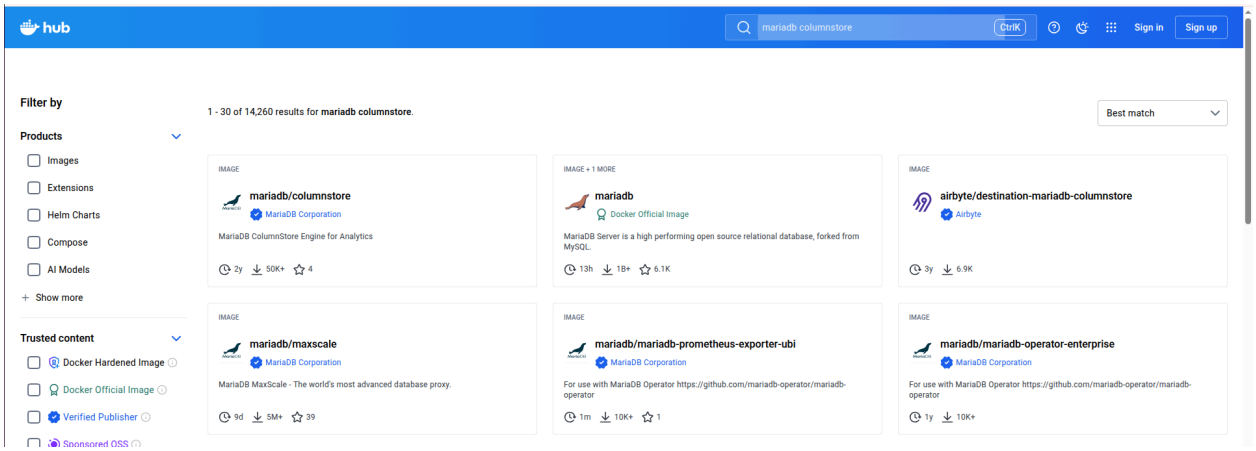
docker info

docker images

docker ps (ps:process status/See running containers)

docker ps -a (See all containers)

Pull image(mariadb/columnstore) from the hub.docker.com




```
docker pull mariadb/columnstore

docker run -d -p 3307:3306 --name mymcs mariadb/columnstore

docker ps -a

docker exec mymcs mariadb -e "GRANT ALL PRIVILEGES ON *.* TO
'mariadbuser'@'%' IDENTIFIED BY 'Sunway@123';"

docker exec mymcs mariadb -e "GRANT ALL PRIVILEGES ON *.* TO
'mariadbuser'@'localhost' IDENTIFIED BY 'Sunway@123';"


docker exec -it mymcs bash

mariadb -u mariadbuser --password='Sunway@123'

SHOW databases;

select user,host from mysql.user;
```

exit

exit

wget https://repo.anaconda.com/miniconda/Miniconda3-py38_4.12.0-Linux-x86_64.sh

bash Miniconda3-py38_4.12.0-Linux-x86_64.sh

source ~/.bashrc

conda –version

conda create --name myfirstenvnt python=3.8 pandas pymysql sqlalchemy jupyter

conda env list

conda activate myfirstenvnt

wget https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data

wget https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv

pip install gdown

```
gdown https://drive.google.com/uc?id=1N4HUduC5PrCflcSD33K7coCA2nyqOTdU
-O FirstTask.ipynb

jupyter notebook
```

Run FirstTask.ipynb from the jupyter notebook

Installation of apache airflow on conda environment:

```
conda create -y --name pipeline_ingestion python=3.10
conda env list
conda activate pipeline_ingestion

pip install "apache-airflow==2.10.5" --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-2.10.5/c
onstraints-3.10.txt"
```

pip install pandas==1.5.3 numpy==1.24.4 SQLAlchemy==1.4.54 pymysql

```
pip list | grep -E "pandas|numpy|SQLAlchemy|PyMySQL"

pip list | grep airflow

airflow
airflow --help
airflow db --help
airflow db migrate
airflow db check
airflow users --help
airflow users list
airflow users create --help
```

```
airflow users create \
  --username system_admin \
  --firstname System \
  --lastname Admin \
  --role Admin \
  --email system@airflow.local \
  --password admin123

airflow users list

cat airflow/airflow.cfg | grep dags_folder
airflow info | grep "airflow_home"

mkdir ~/airflow/dags
cp iris.data ~/airflow/
pip install gdown

gdown https://drive.google.com/uc?id=13W4_NvyFQI61QjIDScPHsVUxCRWPXdGz
-O ~/airflow/dags/example1.py

gdown https://drive.google.com/uc?id=1uyTQmx1_guDyVQhs4ovEl3t90Mq_vIdp
-O ~/airflow/dags/example2.py

ls ~/airflow/dags/

airflow dags list
airflow scheduler
```

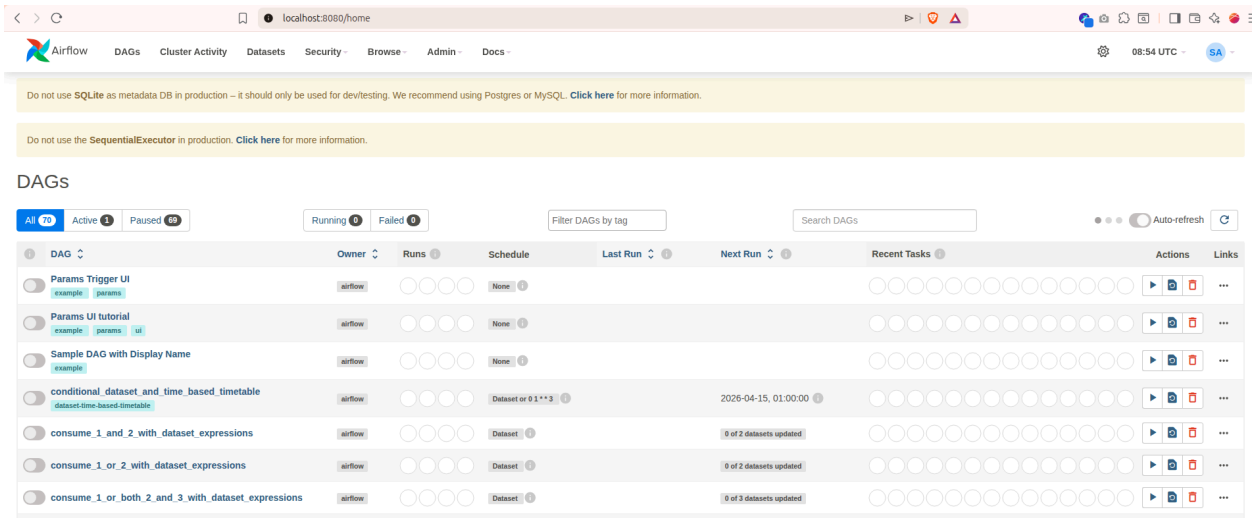
In another terminal, activate your conda environment (pipeline_ingestion) and run

```
conda activate pipeline_ingestion
```

```
airflow webserver
```

Go to the link through the browser:

http://localhost:8080/



After play Trigger DAG check the result:

cat ~/currentdate.txt

cat ~/exampledag1result.txt

cat ~/airflow/duplicateiris.csv

Installation of apache airflow with Docker Image:

```
docker images
docker pull apache/airflow:slim-2.10.5-python3.9

docker run -d \
  -p 9090:8080 \
  --name airflow_container \
  -e AIRFLOW__CORE__EXECUTOR=SequentialExecutor \
  -e
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=sqlite:////opt/airflow/airflow.db
\
```

```
  -e _AIRFLOW_DB_MIGRATE=true \
  -e _AIRFLOW_WWW_USER_CREATE=true \
  -e _AIRFLOW_WWW_USER_USERNAME=admin \
  -e _AIRFLOW_WWW_USER_PASSWORD=admin \
  --entrypoint bash \
  apache/airflow:slim-2.10.5-python3.9 \
  -c "sleep infinity"

docker ps -a
docker ps
docker cp iris.data  airflow_container:/opt/airflow/dags/
```

Need to put both mymcs and airflow_container on the same docker network:

```
docker network ls
docker inspect mymcs | grep Network
docker network create airflow-net
docker network ls
docker network connect airflow-net mymcs
docker network connect airflow-net airflow_container
docker inspect mymcs | grep IPAddress
docker inspect airflow_container | grep IPAddress
docker exec -it -u root airflow_container bash
apt-get update && apt-get install -y nano
apt install iputils-ping -y
ping 172.17.0.2 / ping mcs1
exit
docker exec -it airflow_container bash
ping -c 4 172.17.0.2 / ping -c 4 mymcs
```

airflow --help

airflow db --help

airflow db migrate

```
airflow@e1464eddad0e:/opt/airflow$ airflow db migrate
DB: sqlite:////opt/airflow/airflow.db
Performing upgrade to the metadata database sqlite:////opt/airflow/airflow.db
[2026-04-15T09:15:29.211+0000] {migration.py:207} INFO - Context impl SQLiteImpl.
[2026-04-15T09:15:29.212+0000] {migration.py:210} INFO - Will assume non-transactional DDL.
[2026-04-15T09:15:29.213+0000] {migration.py:207} INFO - Context impl SQLiteImpl.
[2026-04-15T09:15:29.214+0000] {migration.py:210} INFO - Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running stamp_revision  -> 5f2621c13b39
Database migrating done!
airflow@e1464eddad0e:/opt/airflow$ ls
airflow.cfg  airflow.db  dags  logs
```

airflow db check

airflow users --help

airflow users list

```
airflow@e1464eddad0e:/opt/airflow$ airflow users list
No data found
```

airflow users create --help

**airflow users create **

** --username system_admin **

** --firstname System **

** --lastname Admin **

** --role Admin **

** --email system@airflow.local **

** --password admin123**

airflow users list

```
airflow@e1464eddad0e:/opt/airflow$ airflow users list
id | username     | email                | first_name | last_name | roles
===+==============+======================+============+===========+======
1  | system_admin | system@airflow.local | System     | Admin     | Admin
```

pip install gdown

gdown https://drive.google.com/uc?id=13W4_NvyFQI61QjIDScPHsVUxCRWPXdGz -O /opt/airflow/dags/example1.py

gdown https://drive.google.com/uc?id=1P2XONPgZs5cR8AOABFM1iHmGVb5bv_A9 -O /opt/airflow/dags/example2.py

ls /opt/airflow/dags/

airflow dags list

```
airflow@e1464eddad0e:/opt/airflow/dags$ airflow dags list
/home/airflow/.local/lib/python3.9/site-packages/airflow/cli/commands/dag_command.py
raphviz. Rendering graph to the graphical format will not be possible.
dag_id   | fileloc                       | owners       | is_paused
=========+===============================+==============+==========
example1 | /opt/airflow/dags/example.py  | system_admin | False
example2 | /opt/airflow/dags/example1.py | system_admin | False
```

airflow scheduler

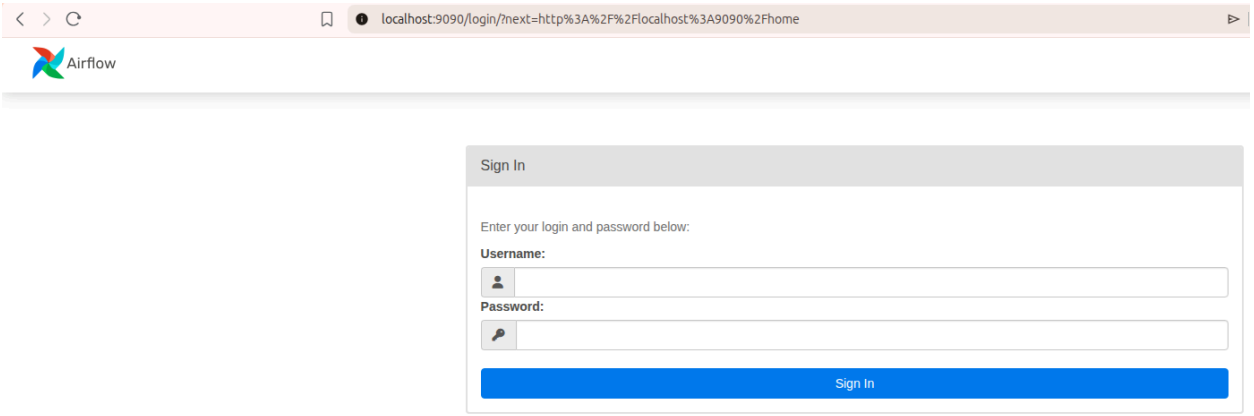
In another terminal

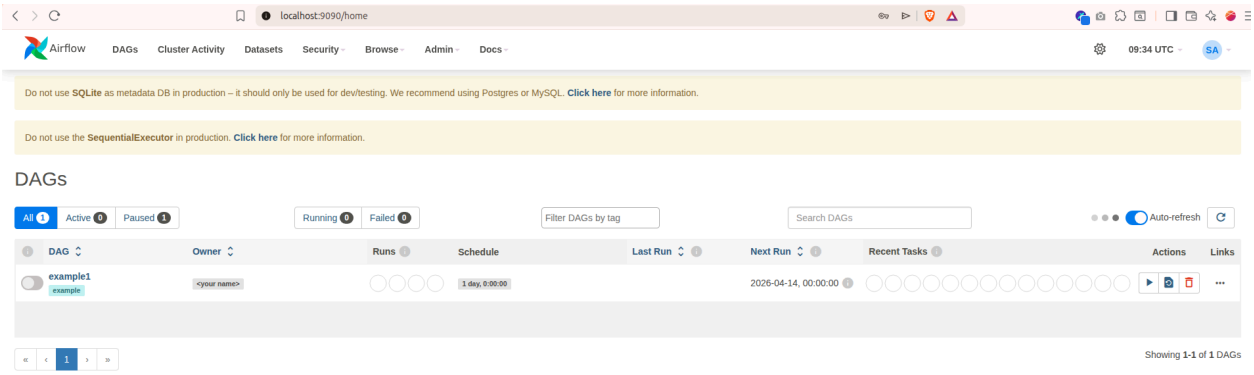
docker exec -it airflow_container bash

airflow webserver

Go to the link through the browser:

http://localhost:9090/





<u>In another terminal (After play Trigger DAG check the result)</u>

docker exec -it airflow_container bash

```
sujan@sujan-Latitude-7490:~$ docker exec -it airflow_container bash
airflow@e1464eddad0e:/opt/airflow$
```

cat /home/airflow/currentdate.txt

cat /home/airflow/exampledag1result.txt

cat /opt/airflow/duplicateiris.csv

cat /home/airflow/currentdate.txt

Installation of radis(Remote Dictionary Server) airflow with Docker Image:

Installation of Redis Server

```
docker pull redis
docker run -d -p 9000:6379 --name myredis redis
docker ps
docker exec -it myredis bash
redis-server --version
redis-cli
set name "Student"
get name
keys *
del name
dbsize
```

Installation of Redis Client:Python Library

In another terminal, activate your conda environment (pipeline_ingestion) and run

```
conda activate pipeline_ingestion
pip install redis
pip install scikit-learn
pip install pyarrow
```

PyArrow = Fast engine that reads/writes big data files efficiently, saves memory, and is the backbone of modern data pipelines. Parquet → fast, small, modern way

```
pip install gdown
gdown https://drive.google.com/uc?id=1jPdpbLaw_3C5sJcFBqiZ72NJRfET5CBd
-O ~/airflow/dags/firstModelPipeline.py

https://drive.google.com/uc?id=1cYqZ61QXCk8GHMwG8eR3rzZLV5abfrjG  -O
Redis.ipynb

pip install jupyter
pip list | grep -E "pandas|numpy|SQLAlchemy|PyMySQL|pyarrow|redis|jupyter"
```

jupyter notebook

- *Run Redis.ipynb file*

In another terminal, activate your conda environment (pipeline_ingestion) and run

`Run airflow scheduler/airflow webserver`

And trigger DAG from http://localhost:8080/

mlFlow

Serialization Libraries

Serialization(Save) = Converting Python objects into bytes that can be saved to disk or sent over network

Deserialization(Load) = Converting those bytes back into Python objects

pickle(Built in): general-purpose

joblib(pip install joblib) : optimized for machine learning

```
conda create -n mlflow_lab python=3.10
conda activate mlflow_lab
conda install -c conda-forge pandas numpy scikit-learn fastapi uvicorn
setuptools mlflow

conda update -n base -c defaults conda
conda update --all

mkdir pickle_model
touch saveModel.py
import pickle
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
```

```
# Load data and train a model
iris = load_iris()
X, y = iris.data, iris.target
model = RandomForestClassifier()
model.fit(X, y)

# Save the model to a file, wb: write binary, rb:read binary
with open("/home/sujan/pickle_model/iris_model.pkl", "wb") as
file:
   pickle.dump(model, file)
print("Model trained and saved to iris_model.pkl")
```

touch loadModel.py

```
import pickle
from sklearn.datasets import load_iris

# Load the model from the file, rb:read file
with open("/home/sujan/pickle_model/iris_model.pkl", "rb") as
file:
   loaded_model = pickle.load(file)
# Load iris data again for sample
iris = load_iris()
X, y = iris.data, iris.target

# Make a prediction using the loaded model
sample_data = X[:1]  # Take the first sample
prediction = loaded_model.predict(sample_data)

print("Sample data:", sample_data[0])
print("Predicted class:", prediction[0])
print("Actual class:", y[0])
```

```
python saveModel.py
```

Python loadModel.py

mkdir fastapiexample

cd fastapiexample

```
(mlflow_lab) sujan@sujan-Latitude-7490:~/fastapiexample1$
```

touch main.py

```
from typing import Union
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"Hello": "World"}
#/items/10
@app.get("/items/{item_id}")
#/items/10?q=fastapi
def read_item(item_id: int, q: Union[str, None] = None):
    return {"item_id": item_id, "q": q}
@app.post("/items/{item_id}")
def create_item(item_id: int, q: str = None):
    return {"message": "Item created via POST", "item_id":
item_id, "q": q}
# activity 2
@app.get("/examples2/{x}/{y}")
def add_numbers(x: int, y: int):
    return {"operation": "sum", "x": x, "y": y, "result": x + y}
#activity 4" multiply
@app.get("/multiply")
def multiply(x: int, y: int):
    return {"result": x * y}
```

uvicorn main:app --reload

http://127.0.0.1:8000/

http://127.0.0.1:8000/items/10?q=fastapi

http://127.0.0.1:8000/examples2/10/23

http://127.0.0.1:8000/multiply?x=10&y=23

Or

curl "http://127.0.0.1:8000/items/10?q=fastapi"

curl -X POST "http://127.0.0.1:8000/items/10?q=fastapi"

nano iris_api.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pandas as pd
import pickle
import numpy as np
# Initialize FastAPI app
app = FastAPI(title="Iris Prediction API")

# Load dataset
file_path = "/home/sujan/iris.data"
columns = ["sepal_length", "sepal_width", "petal_length", "petal_width",
"species"]
try:
   iris_df = pd.read_csv(file_path, names=columns, header=None)
except Exception as e:
   raise RuntimeError(f"Dataset loading failed: {e}")
# Load trained ML model (GLOBAL VARIABLE)
model_path = "/home/sujan/pickle_model/iris_model.pkl"
try:
   with open(model_path, "rb") as f:
       model = pickle.load(f)
except Exception as e:
```

```
    raise RuntimeError(f"Model loading failed: {e}")

# Request Schema
class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float


# Root endpoint
@app.get("/")
def read_root():
    return {
        "message": "Iris Prediction API is running"
    }

# Dataset endpoint
@app.get("/irisdata/{index_id}")
def get_iris_record(index_id: int):
    try:
        if index_id < 0 or index_id >= len(iris_df):
            raise HTTPException(status_code=404, detail="Index out of
range")
        return {
            "record": iris_df.iloc[index_id].to_dict()
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# Prediction endpoint
@app.post("/predict")
def predict_iris(data: IrisInput):
    try:
        # Convert input into numpy array (model requirement)
        features = np.array([[
            data.sepal_length,
            data.sepal_width,
```

```
            data.petal_length,
            data.petal_width
        ]])
        # Make prediction
        prediction = model.predict(features)[0].item()
        # convert to label
        class_names = ["setosa", "versicolor", "virginica"]
        prediction = class_names[prediction]
        return {
            "input": data.model_dump(),
            "prediction": prediction
        }
    except Exception as e:
        # Return proper API error
        raise HTTPException(status_code=500, detail=str(e))
```

http://127.0.0.1:8000/

http://127.0.0.1:8000/irisdata/0

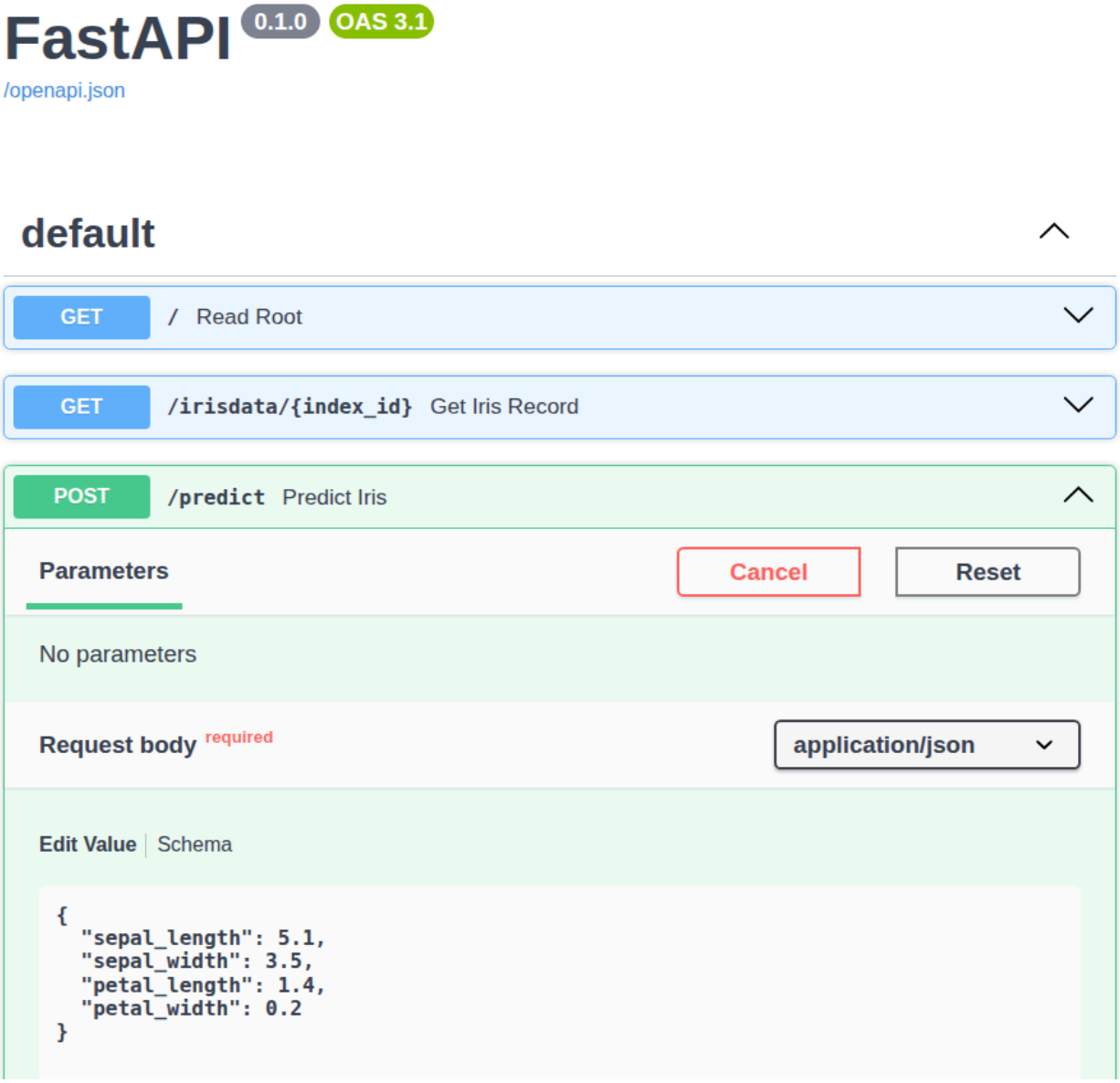
http://127.0.0.1:8000/irisdata/149

http://127.0.0.1:8000/irisdata/-1

http://127.0.0.1:8000/irisdata/150

uvicorn iris_api:app --reload

http://127.0.0.1:8000/docs



Test 1

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

Test 2

```
{
  "sepal_length": 6.2,
  "sepal_width": 2.8,
  "petal_length": 4.8,
  "petal_width": 1.8
}
```

Test 3

```
{
  "sepal_length": 5.6,
  "sepal_width": 2.7,
  "petal_length": 4.2,
  "petal_width": 1.3
}
```

```
mkdir mlflow
cd mlflow
```

```
(mlflow_lab) sujan@sujan-Latitude-7490:~/mlflow$
```

```
mlflow db upgrade sqlite:///mlflow.db
```

```
(mlflow_lab) sujan@sujan-Latitude-7490:~/mlflow$ mlflow db u
pgrade sqlite:///mlflow.db
```

```
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./artifacts \
  --host 127.0.0.1 \
  --port 5000
```

http://127.0.0.1:5000/

mkdir Model

touch mlFlowSaveModel.py

```
import pickle
import mlflow
import warnings
import mlflow.sklearn
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
# Suppress the pickle warning
warnings.filterwarnings("ignore", category=UserWarning, module="mlflow")
# Load data
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Set experiment name
mlflow.set_experiment("iris-classifier")

with mlflow.start_run() as run:
    # Define params
    params = {"n_estimators": 100, "max_depth": 3, "random_state": 42}

    # Train model
    model = RandomForestClassifier(**params)
    model.fit(X_train, y_train)

    # Evaluate
    preds = model.predict(X_test)
```

```
    #  Compute all metrics (average='weighted' for multiclass like Iris)
    acc       = accuracy_score(y_test, preds)
    precision = precision_score(y_test, preds, average="weighted")
    recall    = recall_score(y_test, preds, average="weighted")
    f1        = f1_score(y_test, preds, average="weighted")

    #  Per-class metrics (setosa, versicolor, virginica)
    precision_per_class = precision_score(y_test, preds, average=None)
    recall_per_class    = recall_score(y_test, preds, average=None)
    f1_per_class        = f1_score(y_test, preds, average=None)
    class_names         = iris.target_names  # ['setosa', 'versicolor',
'virginica']

    #  Log params
    mlflow.log_params(params)

    #  Log overall metrics
    mlflow.log_metric("accuracy", acc)
    mlflow.log_metric("precision_weighted", precision)
    mlflow.log_metric("recall_weighted", recall)
    mlflow.log_metric("f1_weighted", f1)

    #  Log per-class metrics
    for i, class_name in enumerate(class_names):
        mlflow.log_metric(f"precision_{class_name}",
precision_per_class[i])
        mlflow.log_metric(f"recall_{class_name}",    recall_per_class[i])
        mlflow.log_metric(f"f1_{class_name}",        f1_per_class[i])

    #  Save classification report as a text artifact
    report = classification_report(y_test, preds, target_names=class_names)
    with open("classification_report.txt", "w") as f:
        f.write(report)
    mlflow.log_artifact("classification_report.txt")

    #  Save confusion matrix as artifact
    cm = confusion_matrix(y_test, preds)
    with open("confusion_matrix.txt", "w") as f:
        f.write("Confusion Matrix (rows=actual, cols=predicted):\n")
```

```
        f.write(f"Classes: {list(class_names)}\n\n")
        f.write(str(cm))
    mlflow.log_artifact("confusion_matrix.txt")

    # Log model to MLflow
    mlflow.sklearn.log_model(model, name="iris_model")

    # Also save as pickle
    with open("/home/sujan/mlflow/Model/iris_model.pkl", "wb") as file:
        pickle.dump(model, file)

    # Print summary
    print(f"\n{'='*45}")
    print(f"  Run ID   : {run.info.run_id}")
    print(f"{'='*45}")
    print(f"  Accuracy : {acc:.4f}")       # ← these must be indented
    print(f"  Precision: {precision:.4f}")  # ← inside the with block
    print(f"  Recall   : {recall:.4f}")
    print(f"  F1 Score : {f1:.4f}")
    print(f"{'='*45}")
    print("Model saved to MLflow and pickle!")
```

touch mlFlowIris_api.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pandas as pd
import numpy as np
import mlflow.pyfunc
import mlflow

# Initialize FastAPI app
app = FastAPI(title="Iris Prediction API")

#  Set the MLflow tracking URI (same as where saveModel.py logs to)
mlflow.set_tracking_uri("http://127.0.0.1:5000")  # default local MLflow
server

#  Load model from MLflow by experiment name (always gets latest run)
def load_latest_model():
```

```
    client = mlflow.tracking.MlflowClient()

    # Get experiment by name
    experiment = client.get_experiment_by_name("iris-classifier")
    if experiment is None:
        raise RuntimeError("Experiment 'iris-classifier' not found in
MLflow.")

    # Get the latest run (sorted by start time)
    runs = client.search_runs(
        experiment_ids=[experiment.experiment_id],
        order_by=["start_time DESC"],
        max_results=1
    )
    if not runs:
        raise RuntimeError("No runs found in experiment
'iris-classifier'.")

    latest_run_id = runs[0].info.run_id
    print(f"Loading model from Run ID: {latest_run_id}")

    # Load the model
    model = mlflow.pyfunc.load_model(f"runs:/{latest_run_id}/iris_model")
    return model, latest_run_id

# Load model at startup (GLOBAL)
try:
    model, active_run_id = load_latest_model()
    print(" Model loaded from MLflow successfully!")
except Exception as e:
    raise RuntimeError(f"Model loading failed: {e}")

# Class labels
CLASS_NAMES = ["setosa", "versicolor", "virginica"]

# Request Schema
class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
```

```
    petal_length: float
    petal_width: float

# Root endpoint
@app.get("/")
def read_root():
    return {
        "message": "Iris Prediction API is running",
        "model_source": "MLflow",
        "run_id": active_run_id
    }

#  Model info endpoint — shows metrics from MLflow
@app.get("/model/info")
def get_model_info():
    try:
        client = mlflow.tracking.MlflowClient()
        run = client.get_run(active_run_id)

        return {
            "run_id": active_run_id,
            "experiment": "iris-classifier",
            "params": run.data.params,
            "metrics": {k: round(v, 4) for k, v in
run.data.metrics.items()},
            "status": run.info.status
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# Prediction endpoint
@app.post("/predict")
def predict_iris(data: IrisInput):
    try:
        #  MLflow pyfunc requires a DataFrame input
        features = pd.DataFrame([[
            data.sepal_length,
            data.sepal_width,
            data.petal_length,
```

```
            data.petal_width
        ]], columns=["sepal_length", "sepal_width", "petal_length",
"petal_width"])

        # Make prediction
        prediction_idx = int(model.predict(features)[0])
        prediction_label = CLASS_NAMES[prediction_idx]

        return {
            "input": data.model_dump(),
            "prediction": prediction_label,
            "run_id": active_run_id
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

#  Reload model endpoint — pull the latest run without restarting API
@app.post("/model/reload")
def reload_model():
    global model, active_run_id
    try:
        model, active_run_id = load_latest_model()
        return {
            "message": "Model reloaded successfully!",
            "new_run_id": active_run_id
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

```
(mlflow_lab) sujan@sujan-Latitude-7490:~/mlflow$ touch mlFlowSaveModel.py
(mlflow_lab) sujan@sujan-Latitude-7490:~/mlflow$ touch mlFlowIris_api.py
```

python mlFlowSaveModel.py

mlflow experiments search

uvicorn mlFlowIris_api:app --reload